COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	August 5, 2026	**DATE SUBMITTED:**	August 7, 2026
NAME:	Julian Benedict Barizo	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 5 **Queues**

1. PROCEDURE OUTPUT

ILO A : STL QUEUE

main.cpp

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> copyQ);

int main() {

    // Create an object
    std::queue<char> myQ;

    // Use Enqueue operation
    myQ.push('M');
    myQ.push('A');
    myQ.push('R');
    myQ.push('I');
    myQ.push('E');


    // Print the front and the back of the queue
    std::cout << "Current front of the queue is: " << myQ.front() << std::endl;
    std::cout << "Current back of the queue is: " << myQ.back() << std::endl;
    std::cout << std::endl;
    // Print the ENTIRE queue
    display(myQ);

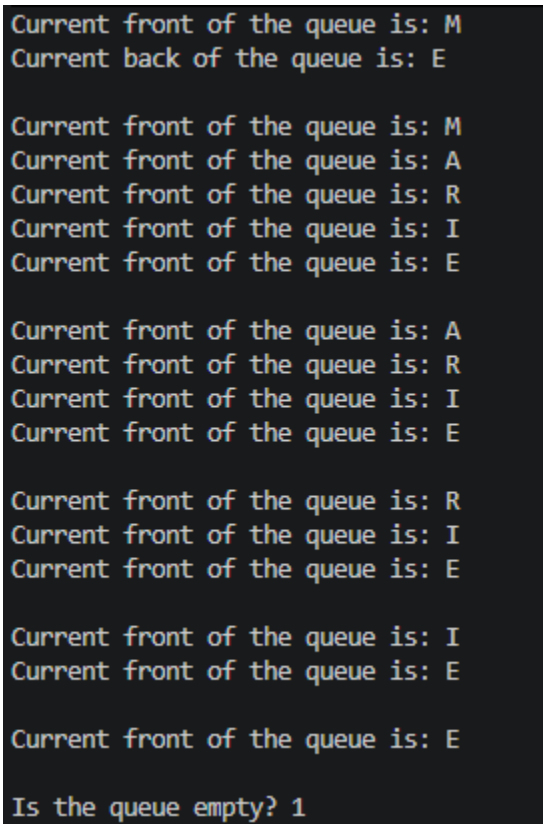
    // Use Dequeue operation
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();


    std::cout << "Is the queue empty? " << myQ.empty() << std::endl;

    return 0;
}
```

```cpp
void display(std::queue<char> copyQ){

    // Create a copy of the queue
    std::queue<char> tempQ(copyQ);
    // Loop until Empty
    while (!tempQ.empty()) {
        std::cout << "Current front of the queue is: " << tempQ.front() << std::endl;
        tempQ.pop();
    }
    std::cout << std::endl;
}
```

	Observations: The code demonstrates the implementation of a standard C++ First-In, First-Out (FIFO) queue using std::queue<char> to store and manage characters. It adds the given letters to the queue, displays the first and last elements, and uses a custom function to show all the elements without altering the queue itself. The program then removes each element one at a time using pop(), displaying the remaining contents after every removal. Finally, it verifies that the queue is empty after all elements have been removed.
Output	
	Observations: The program demonstrates the use of a C++ queue by adding characters one after another and displaying the elements at the front and back. A helper function is used to create a copy of the queue so its contents can be displayed without changing the original queue. The elements are then removed individually in sequence, and the program ends by checking whether the queue is empty and displaying a Boolean value to confirm that all elements have been removed.

ILO B1 : QUEUE LINKED LIST

main.cpp

```cpp
#include <iostream>
#include "queueLL.h"

int main() {

    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    std::cout << "===== TESTING ENQUEUE =====\n";

    enqueue('B', &front, &back);
    enqueue('E', &front, &back);
    enqueue('N', &front, &back);
    enqueue('D', &front, &back);
    enqueue('Y', &front, &back);

    displayALL(&front, &back);

    std::cout << "Front: " << front->data << std::endl;
    std::cout << "Back : " << back->data << std::endl;

    std::cout << "\nIs Empty? "
              << (isEmpty(&front, &back) ? "Yes" : "No")
              << std::endl;

    std::cout << "\n===== TESTING DEQUEUE =====\n";

    dequeue(&front, &back);
    displayALL(&front, &back);

    dequeue(&front, &back);
    displayALL(&front, &back);

    dequeue(&front, &back);
    displayALL(&front, &back);
```

```
    dequeue(&front, &back);
    displayALL(&front, &back);

    dequeue(&front, &back);
    displayALL(&front, &back);

    std::cout << "\nIs Empty? "
              << (isEmpty(&front, &back) ? "Yes" : "No")
              << std::endl;

    // Test dequeue on empty queue
    dequeue(&front, &back);

    return 0;
}
```

queueLL.h

```h
#ifndef QUEUELL_H
#define QUEUELL_H

#include <iostream>

template <typename T>
class qNode {
public:
    T data;
    qNode* next;
};

// Create a new node
template <typename T>
qNode<T>* new_node(T newData) {
    qNode<T>* newNode = new qNode<T>();
    newNode->data = newData;
    newNode->next = nullptr;

    return newNode;
}

// Enqueue
template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr) {

    qNode<T>* newNode = new_node(newData);

    if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
        return;
    }

    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
}
```

```cpp
// Dequeue
template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr) {

    if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
        std::cout << "The queue is empty." << std::endl;
        return;
    }

    qNode<T>* deleteTemp = *frontPtr;

    // Only one node
    if ((*frontPtr)->next == nullptr) {
        *frontPtr = nullptr;
        *backPtr = nullptr;
        delete deleteTemp;
        return;
    }

    *frontPtr = (*frontPtr)->next;
    delete deleteTemp;
}

// Check if queue is empty
template <typename T>
bool isEmpty(qNode<T>** frontPtr, qNode<T>** backPtr) {
    return ((*frontPtr) == nullptr && (*backPtr) == nullptr);
}
```

```cpp
// Display all elements
template <typename T>
void displayALL(qNode<T>** frontPtr, qNode<T>** backPtr) {

    if (isEmpty(frontPtr, backPtr)) {
        std::cout << "Queue is empty." << std::endl;
        return;
    }

    qNode<T>* temp = *frontPtr;

    std::cout << "Queue Elements: ";

    while (temp != nullptr) {
        std::cout << temp->data << " ";
        temp = temp->next;
    }

    std::cout << std::endl;
}

#endif
```

Output

```
===== TESTING ENQUEUE =====
Queue Elements: B E N D Y
Front: B
Back : Y

Is Empty? No

===== TESTING DEQUEUE =====
Queue Elements: E N D Y
Queue Elements: N D Y
Queue Elements: D Y
Queue Elements: Y
Queue is empty.

Is Empty? Yes
The queue is empty.
```

Observations

Through implementing a queue using a linked list, I was able to better visualize how elements are added at the back and removed from the front while following the FIFO (First In, First Out) principle. Testing each operation also helped me understand how the front and back pointers

	change as elements are added and removed until the queue becomes empty. Functions such as enqueue(), displayALL(), isEmpty(), and dequeue() demonstrate the current state of the queue and how each operation affects it. The displayALL() function allows me to view the elements from front to back, while isEmpty() checks whether the queue still contains any data. Overall, this activity helped me understand how the different functions work together to manage the queue and how the front and back pointers are updated after every operation.

ILO B2 : QUEUE ARRAY

main.cpp

```cpp
int main() {
    //creating an object
    queueArr<int> q(5);

    ///enqeueue 1,2,3,4
    q.Enqueue(1);

    std::cout<<"Front: " << q.front() << std::endl;
    std::cout<<"Back: " << q.back() << std::endl;
    std::cout<<"Size: " << q.Size() << std::endl;


    q.Enqueue(2);

    std::cout<<"Front: " << q.front() << std::endl;
    std::cout<<"Back: " << q.back() << std::endl;
    std::cout<<"Size: " << q.Size() << std::endl;

    q.Enqueue(3);

    std::cout<<"Front: " << q.front() << std::endl;
    std::cout<<"Back: " << q.back() << std::endl;
    std::cout<<"Size: " << q.Size() << std::endl;

    q.Enqueue(4);

    std::cout<<"Front: " << q.front() << std::endl;
    std::cout<<"Back: " << q.back() << std::endl;
    std::cout<<"Size: " << q.Size() << std::endl;

    q.Enqueue(5);

    std::cout<<"Front: " << q.front() << std::endl;
    std::cout<<"Back: " << q.back() << std::endl;
    std::cout<<"Size: " << q.Size() << std::endl;
```

```
    q.Enqueue(6);

    std::cout << "removed: " << q.Dequeue() << std::endl;
    std::cout << "removed: " << q.Dequeue() << std::endl;
    std::cout << "removed: " << q.Dequeue() << std::endl;

    //testing clear
    q.Clear();
    std::cout<<"Is the queue empty? " << (q.Empty() ? "yes": "no") << std::endl;

    return 0;
}
```

queueArr.h

```
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>


//Array-Based Circular Queue
template <typename T>
class queueArr{
    private:
    //pointer to dynamically allocate an array
        T* q_array;


    //variable for the maximum number of elements a quueue can hold
        size_t q_capacity;

    //current number of the elements in the queue
        size_t q_size;

    //Index of the front element
        int q_front;

    //Index of the back element
        int q_back;
```

```
    public:
        //constructor
        queueArr(size_t capacity = 10);


        //copy constructor
        queueArr(const queueArr& other);

        //copy assignment operator
        queueArr& operator=(const queueArr& other);

        //destructor
        ~queueArr();

        //queue operations
        bool Empty();
        bool Full();


        size_t Size();

        void Clear();


        T front();
        T back();

        void Enqueue(T value);
        T Dequeue();

};
```

```cpp
//constructor
template <typename T>
queueArr<T>::queueArr(size_t capacity) {
    //capacity, q_size, q_front, q_back and q_array
    q_capacity = capacity;
    q_size = 0;
    q_front = 0;
    q_back = -1;
    q_array = new T[q_capacity];

    //size, front, back, and array
}


//copy constructor
template <typename T>
queueArr<T>::queueArr(const queueArr& other){
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;
    q_array = new T[q_capacity];
    for(size_t i = 0; i < q_capacity; i++){
        q_array[i] = other.q_array[i];
    }
}
```

```cpp
//copy assignment operator
template <typename T>
queueArr<T>& queueArr<T>::operator=(const queueArr& other){
    if(this != &other){
        delete[] q_array;
        q_capacity = other.q_capacity;
        q_size = other.q_size;
        q_front = other.q_front;
        q_back = other.q_back;
        q_array = new T[q_capacity];
        for(size_t i = 0; i < q_capacity; i++){
            q_array[i] = other.q_array[i];
        }
    }
    return *this;
}


//destructor
template <typename T>
queueArr<T>::~queueArr(){
    delete[] q_array;
}

//Empty()
template <typename T>
bool queueArr<T>::Empty(){
    ///check if the size is 0
    return q_size == 0;


}
```

```cpp
//Full()
template <typename T>
bool queueArr<T>::Full(){
    //check if the queue size reached the max capacity
    return q_size == q_capacity;
}

//Find the current size of the array
//size_T
template <typename T>
    size_t queueArr<T>::Size(){
        return q_size;
    }

//front:
template <typename T>
T queueArr<T>::front(){
    //check if the queue is empty
    if(Empty()) {
        std::cout<<"Queue is Empty\n";
        return T{};
    }
    //return the front
    return q_array[q_front];
}
```

```cpp
//back:
template <typename T>
T queueArr<T>::back(){
    //Check if the queue is empty
    if(Empty()){
        std::cout<<"Queue is empty"<<std::endl;
    return T{};
}
    //return to the back
        return q_array[q_back];
}

//clear
template<typename T>
void queueArr<T>::Clear(){

    //reset the q_size, q_front, q_back
    q_size = 0;
    q_front = 0;
    q_back = -1;

}
```

```cpp
template <typename T>
void queueArr<T>::Enqueue(T value){

    //check if the queue is full
    if(Full()){
        std::cout<<"Queue is full\n";
        return;
    }


    //move q_back circularly
    q_back = (q_back + 1) % q_capacity;


    //store the value to the array
    q_array[q_back] = value;

    //increment the q_size
    q_size++;
}
```

```cpp
//T Dequeue();
template <typename T>
T queueArr<T>::Dequeue(){
    //check if empty
    if (Empty()) {
        std::cout << "Qeueue is currently empty.\n";
        return T{};
    }

    //create a temporary variable to store the current front
    T temp = q_array[q_front];

    //move q_front circularly
    q_front = (q_front + 1) % q_capacity;

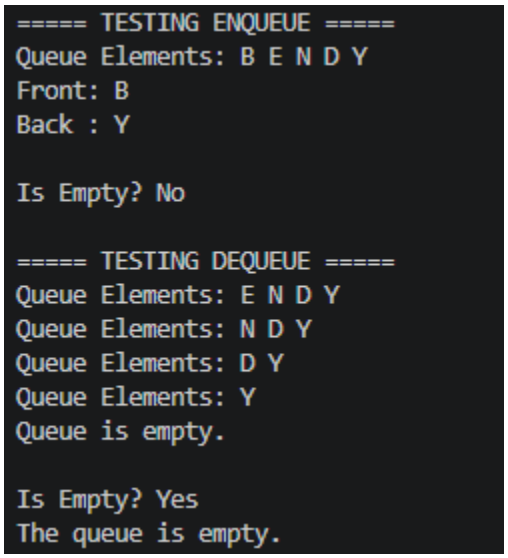
    //decrement the q_size
    q_size -- ;

    //reset the indexes if the queue becomes empty;
    if (Empty()) Clear();


    //return the temporary variable
    return temp;

}


#endif
```

Output	
Observations	The copy constructor and copy assignment operator helped me visualize how a queue can create its own separate copy without sharing the same memory. The copy constructor creates a new array and copies each element from the original queue, while the copy assignment operator first releases the previously allocated memory before creating a new array and copying the updated data. This helped me better understand how dynamic memory is managed safely and why each queue maintains its own independent copy without affecting the other.

2. ANSWERS TO SUPPLEMENTARY ACTIVITY

ILO C : Solve Problems Using Queue Implementations

FILES

Job.h

HEADER

```
#ifndef JOB_H
#define JOB_H

#include <string>
#include <iostream>

// Represents a single print job: who submitted it, its ID, and how many
// pages it needs printed.
class Job {
private:
    int jobID;
    std::string userName;
    int pageCount;

public:
    Job() : jobID(0), userName(""), pageCount(0) {}

    Job(int id, const std::string& name, int pages)
        : jobID(id), userName(name), pageCount(pages) {}

    int getJobID() const { return jobID; }
    std::string getUserName() const { return userName; }
    int getPageCount() const { return pageCount; }

    void print() const {
        std::cout << "Job #" << jobID
                  << " | Submitted by: " << userName
                  << " | Pages: " << pageCount << "\n";
    }
};

#endif
```

Observations: This header defines the Job class, a lightweight data holder representing a single print request: its ID, the submitting user's name, and page count. It offers only getters and a print() method for formatted output, with no queue logic or processing behavior of that responsibility belonging entirely to Printer. This file represents the individual unit of work: each Job instance is one ticket sitting in the printer's queue, waiting its turn.

Printer.h

```h
#ifndef PRINTER_H
#define PRINTER_H

#include <iostream>
#include "Job.h"


class Printer {
private:
    Job* jobs;
    int capacity;
    int count;   // number of jobs currently waiting
    int front;   // index of the next job to be processed

    void grow() {
        int newCapacity = capacity * 2;
        Job* newJobs = new Job[newCapacity];
        for (int i = 0; i < count; i++) {
            newJobs[i] = jobs[(front + i) % capacity];
        }
        delete[] jobs;
        jobs = newJobs;
        capacity = newCapacity;
        front = 0;
    }

public:
    Printer(int startingCapacity = 4)
        : capacity(startingCapacity), count(0), front(0) {
        if (capacity < 1) capacity = 1;
        jobs = new Job[capacity];
    }

    ~Printer() {
        delete[] jobs;
    }
```

```cpp
// Step 2: interface to add a new job to the back of the queue
void addJob(const Job& job) {
    if (count == capacity) {
        grow();
    }
    int rear = (front + count) % capacity;
    jobs[rear] = job;
    count++;
    std::cout << "Added -> ";
    job.print();
}

bool isEmpty() const {
    return count == 0;
}

// Step 3: FCFS -- always process the job that has been waiting longest,
// i.e. the one at the front of the array-backed queue.
bool processNextJob() {
    if (isEmpty()) {
        std::cout << "No jobs to process. Printer is idle.\n";
        return false;
    }
    Job current = jobs[front];
    std::cout << "Printing -> ";
    current.print();
    front = (front + 1) % capacity;
    count--;
    return true;
}
```

```
    void processAllJobs() {
        std::cout << "\n--- Printer starting to process queue ---\n";
        while (!isEmpty()) {
            processNextJob();
        }
        std::cout << "--- All jobs completed ---\n\n";
    }

    void showQueue() const {
        if (isEmpty()) {
            std::cout << "Queue is empty.\n";
            return;
        }
        std::cout << "\nJobs waiting (front to back):\n";
        for (int i = 0; i < count; i++) {
            std::cout << "  ";
            jobs[(front + i) % capacity].print();
        }
    }
};

#endif
```

Observations: This header defines the Printer class, which manages the shared print queue using a plain array with front and count indices instead of any STL container. It exposes addJob() to enqueue new print requests to the back of the array and processNextJob()/processAllJobs() to serve them strictly in the order they arrived, honoring first-come-first-served. A private grow() helper resizes the underlying array when it fills up, so the printer's capacity isn't fixed. This file represents the printer, the machine that holds pending work and decides what gets printed next.

MAIN DRIVER

main.cpp

```cpp
#include "Printer.h"

// with the printer processing them first come, first served.
int main() {
    Printer officePrinter(4);

    std::cout << "=== Shared Printer Simulation ===\n\n";

    // A batch of users submit jobs before the printer starts working
    officePrinter.addJob(Job(101, "Elijah", 5));
    officePrinter.addJob(Job(102, "Jesse", 12));
    officePrinter.addJob(Job(103, "Cedric", 3));

    officePrinter.showQueue();

    // Printer works through the queue in the order jobs arrived
    officePrinter.processNextJob();

    // More users submit jobs while the printer is still busy with the rest
    officePrinter.addJob(Job(104, "Ryle", 8));
    officePrinter.addJob(Job(105, "Thomas", 1));

    officePrinter.showQueue();

    // Finish off everything remaining in FCFS order
    officePrinter.processAllJobs();

    std::cout << "Printer status: "
              << (officePrinter.isEmpty() ? "idle, no jobs left" : "still has jobs")
              << "\n";

    return 0;
}
```

Observations:This is the driver file that simulates real office activity: several users submitting jobs at different points in time, some arriving while the printer is already working through earlier ones. It exercises every public method on Printer adding jobs, displaying the current queue, and processing jobs one at a time or all at once to demonstrate that the FCFS ordering holds up as new jobs are inserted mid-run. This file represents the simulation itself, the scenario that puts the Job and Printer classes to work together.

OUTPUT

```
=== Shared Printer Simulation ===

Added -> Job #101 | Submitted by: Elijah | Pages: 5
Added -> Job #102 | Submitted by: Jesse | Pages: 12
Added -> Job #103 | Submitted by: Cedric | Pages: 3

Jobs waiting (front to back):
  Job #101 | Submitted by: Elijah | Pages: 5
  Job #102 | Submitted by: Jesse | Pages: 12
  Job #103 | Submitted by: Cedric | Pages: 3
Printing -> Job #101 | Submitted by: Elijah | Pages: 5
Added -> Job #104 | Submitted by: Ryle | Pages: 8
Added -> Job #105 | Submitted by: Thomas | Pages: 1

Jobs waiting (front to back):
  Job #102 | Submitted by: Jesse | Pages: 12
  Job #103 | Submitted by: Cedric | Pages: 3
  Job #104 | Submitted by: Ryle | Pages: 8
  Job #105 | Submitted by: Thomas | Pages: 1

--- Printer starting to process queue ---
Printing -> Job #102 | Submitted by: Jesse | Pages: 12
Printing -> Job #103 | Submitted by: Cedric | Pages: 3
Printing -> Job #104 | Submitted by: Ryle | Pages: 8
Printing -> Job #105 | Submitted by: Thomas | Pages: 1
--- All jobs completed ---
```

Why Queue Array?

Using an array-based queue made it easier for me to understand how elements are stored in consecutive memory locations. I noticed that adding and removing elements is efficient when they follow the normal FIFO order, but removing an element from the middle requires the remaining elements to shift. This helped me understand why array-based queues are effective for regular queue operations but can be less practical when frequent middle deletions are required. I also implemented these concepts to further improve my understanding, especially since I found today's activity quite challenging. Despite the difficulty, working through the different operations helped me gain a better understanding of how array-based queues manage and organize data.

CONCLUSION

This activity helped me better understand how a circular array queue works by simulating a real-world printer queue. Through this simulation, I was able to see how jobs are added, processed, and removed in the proper FIFO order, making it easier to visualize how queue operations are applied in actual situations. It also helped me

understand how a circular queue can reuse available spaces in the array instead of leaving unused spaces behind. Overall, the exercise improved my understanding of queue implementation and gave me a clearer idea of the strengths and limitations of using an array-based queue.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.